American University of Sharjah
College of Engineering
Department of Computer
Science and Engineering
P. O. Box 26666, Sharjah, UAE

Instructor: Dr. Mustafa Hammad
Lab Instructor: Khaldoon Alhusari
Office: EB2-118 (inside EB2-117)
Phone: +971 6 515 4819
E-mail: kalhusari@aus.edu
Semester: Fall 2026

COE420L - Software Engineering - Lab 4

Data Flow Modeling and Project Scheduling

Objectives:

By completing this lab, students will be able to:

1. Develop a Context Diagram and Level-0 Data Flow Diagram (DFD) for a proposed software system.
2. Apply DFD modeling principles to represent external entities, processes, data flows, and data stores.
3. Develop a Work Breakdown Structure (WBS) based on the actual development activities of the project.
4. Develop a Gantt Chart and network-based project schedule using project activities and dependencies.
5. Apply Critical Path Method (CPM) concepts to calculate project scheduling parameters and identify the Critical Path.

Due date: Start of next-next week's lab.

- A **short oral discussion** with the lab instructor will be conducted during **next week's lab to discuss the progress**. Each team should be prepared to explain the work completed so far and discuss any issues or decisions related to this lab's work. Note that presenting work on a tablet or laptop is *explicitly preferred*.
- Only one submission per team. Do not forget to include the AUS IDs and names of team members in the lab report.

Note: Using ChatGPT (or any other LLMs/AI tools) without proper declaration and justification will be considered a violation of the AUS integrity code.

General Instructions:

- This lab is a **team-based activity**.
- Continue working on the same project and GitHub repository used in Labs 1-3.
- All models and schedules must be based on the team's **actual proposed software system**.
- Students must use an appropriate software tool to create the DFDs, Gantt Chart, and network diagram. Hand-drawn diagrams are not acceptable.
- The team may use any suitable modeling, diagramming, or project-management tool.
- All diagrams and tables must be clearly labeled and professionally organized.

- The work must be consistent with the requirements, Use Cases, actors, and project scope developed in previous labs.
- The team should maintain meaningful GitHub commits that demonstrate team participation.

Submission:

- A full report with responses to all tasks, with team member names and IDs clearly highlighted.
- A link to the GitHub repository created in Lab 1

Exercise 1: Context Diagram

Develop a **Context Diagram** for the proposed software system.

The Context Diagram provides a high-level view of the system by representing the entire proposed software system as **one process** and showing its interaction with external entities.

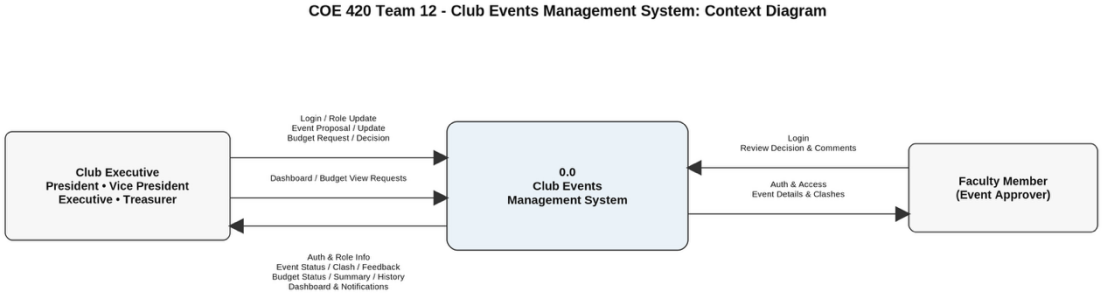
The diagram should include:

- The proposed system as one central process.
- All relevant external entities.
- Data flows entering the system.
- Data flows leaving the system.
- Clear names for all entities and data flows.

Requirements

The Context Diagram must:

1. Use the project's actual actors/stakeholders.
2. Be consistent with the UML Use Case Diagram from Lab 3.
3. Clearly identify incoming and outgoing data flows.
4. Avoid including internal processes or internal data stores.
5. Be created using a suitable diagramming/modeling tool.



Exercise 2: Level-0 Data Flow Diagram

Develop a **Level-0 DFD** that decomposes the system represented in the Context Diagram into its major processes.

The Level-0 DFD must include:

- The same external entities represented in the Context Diagram.
- At least **5 major processes**.
- At least **2 data stores**.
- Data flows between external entities, processes, and data stores.
- Clearly named processes, data flows, and data stores.

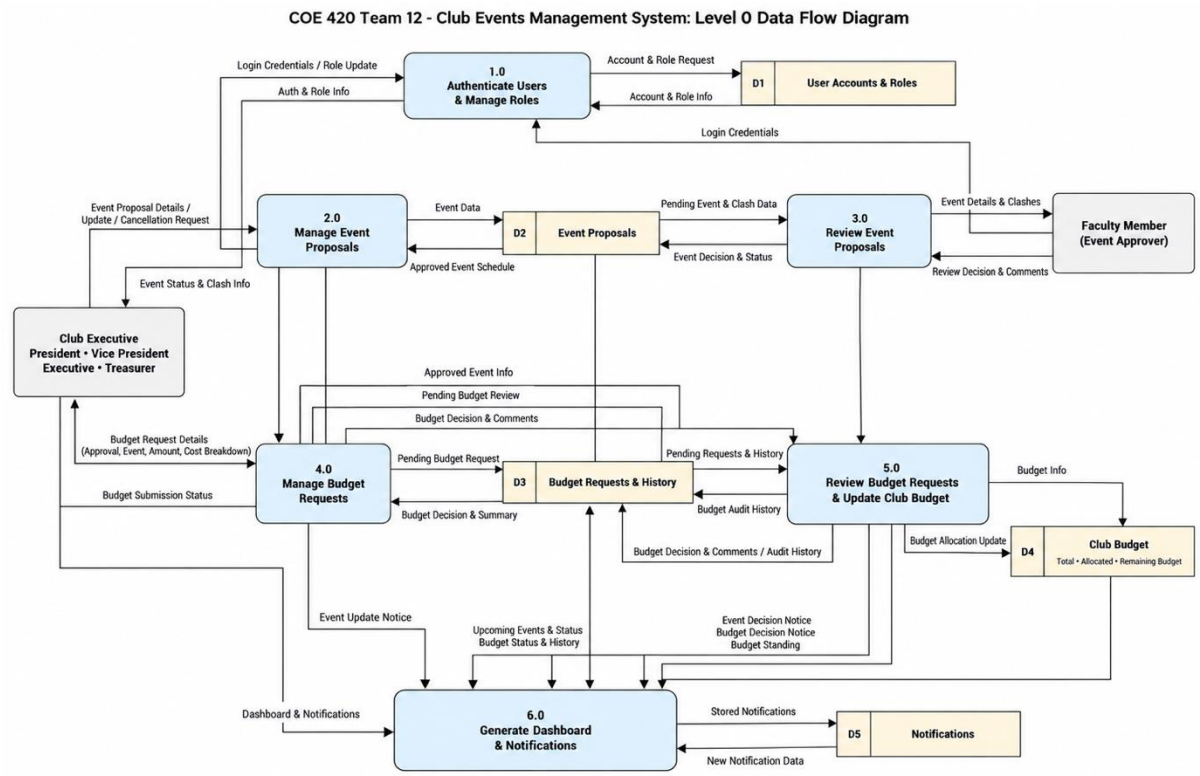
Important

1. The Level-0 DFD should cover the major data-processing aspects of the project.
2. The processes should be derived from the team's:

    - Functional Requirements
    - Use Cases
    - Scenarios
    - Project scope

3. Do not create processes simply to satisfy the minimum number. Each process should represent a meaningful part of the proposed system.

✓ **DFD Balancing and Consistency :** The Context Diagram and Level-0 DFD must be **balanced**. This means that the external inputs and outputs shown in the Context Diagram must be represented consistently in the Level-0 DFD. For example, if an external entity sends **Customer Information** to the system in the Context Diagram, the Level-0 DFD should show how that information enters an appropriate process. Similarly, information leaving the system in the Level-0 DFD should correspond to the outputs represented at the Context Diagram level.



Exercise 3: Core Processes Matrix

Prepare a **Core Processes Matrix** describing the major processes identified in the Level-0 DFD. The processes in this matrix must be the **same major processes represented in the Level-0 DFD**.

At least **5 main processes** are required.

Required Format

Process No. and Name	Input Data	Output Data	Rules
P1 -Authenticate Users & Manage Roles	Login credentials, role updates, stored account information	Authentication results, role information, updated account records	Validate credentials against stored user accounts. Apply role changes only when permitted.
P2 -Manage Event Proposals	New event proposals, event updates,	Stored event proposals, event status,	Check required event information. Save new or updated proposals for review.

	existing event information	clash information, event update notices	
P3 -Review Event Proposals	Pending proposals, clash data, faculty review decisions and comments	Event approval status, review feedback, event decision notifications	Faculty approver reviews proposals and clash information. Record decisions and update event status.
P4 -Manage Budget Requests	Budget request details, relevant event information	Pending budget requests, submission status, request history	Validate request details, register submissions, and maintain their pending status until a decision is made.
P5 -Review Budget Requests & Update Club Budget	Pending requests, request history, budget information, budget decisions	Budget decisions, updated allocations, audit history, budget summaries	Check remaining funds before approving allocations. Record decisions and update allocated and remaining budgets accordingly.
P6-Generate Dashboard & Notifications	Dashboard requests, event updates, budget status, decisions, notification history	Dashboard information, event and budget notifications, stored notification records	Display relevant information according to user access. Generate notifications when event or budget statuses change.

Add additional processes if required by the project.

Process Rules

For each process, explain the major rules or conditions that govern how the input data is transformed into output data. The rules should be specific to the proposed system.

Exercise 4: Develop the Work Breakdown Structure (WBS)

A **Work Breakdown Structure (WBS)** divides a project into smaller, manageable activities. For this lab, the WBS should represent the actual work required to develop the team's proposed software system. The WBS should be based on the team's project rather than a generic list such as:

Analysis → Design → Coding → Testing

Students should identify meaningful project activities related to their own system. Depending on the project, activities may include:

- Requirements refinement
- Database design
- User-interface design
- Authentication implementation
- Specific module development
- Integration
- Testing
- Deployment preparation

✓ The activities should be sufficiently detailed to support project scheduling.

Develop a WBS containing at least 6 major activities required to complete the proposed project. For each activity, provide:

- Task ID
- Task Name
- Description
- Predecessor(s)
- Time Duration in days

WBS:

ID	Task Name	Description	Predecessor(s)	Duration (Days)
T1	**Requirements Refinement & System Design**	Finalize roles, statuses and rules. Design the database schema. Set up the repository.	-	7
T2	**Authentication & Role Management**	Secure AUS email login, server-side role-based access checks, and assigning/changing club roles (FR1, FR2, FR5).	T1	5

T3	**Event Management Module**	Event proposals with validation and routing, room clash check, faculty approve/reject/request changes, resubmission and cancellation (FR11-FR15).	T2	10
T4	**Budget Management Module**	Budget requests for approved events, President approval with audit log, remaining-balance updates, budget summary and history (FR6-FR10).	T2	8
T5	**Notifications & Role-Based Dashboard**	Notifications for event and budget decisions/changes, and the role-based dashboard of events, statuses and budget (FR3, FR4).	T3, T4	5
T6	**Integration, Testing & Deployment**	Integrate all modules, run system and regression tests, fix defects, deploy the web app and write the user guide.	T5	6

The WBS must:

- Represent the actual project.
- Include logical dependencies.
- Use realistic durations.
- Include enough activities to represent the project meaningfully.
- Provide sufficient information to construct the Gantt Chart and network schedule.

Exercise 5: Gantt Chart

Using the WBS developed in Exercise 4, create a **Gantt Chart** for the project.

The Gantt Chart should visually represent:

- Project activities
- Start and finish dates or periods
- Activity durations
- Dependencies if applicable
- Overall project timeline

✓ The Gantt Chart must be created using a suitable project-management or scheduling tool.

Important

The activities in the Gantt Chart must correspond directly to the activities in the WBS. Do not create a separate set of activities for the Gantt Chart.

Tool: mermaid.ink

Start date Monday 19 Oct 2026. Mon-Fri working days.

Red bars are critical tasks.

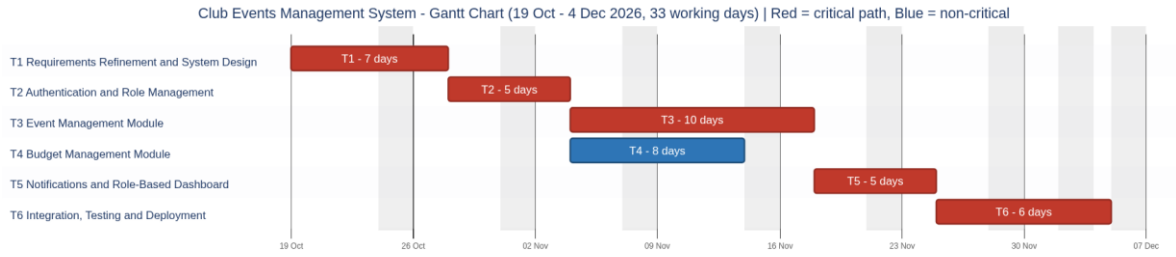


Figure 5: Gantt Chart

ID	Task	Start	Finish	Days
T1	Requirements Refinement & System Design	Mon 19 Oct 2026	Tue 27 Oct 2026	7
T2	Authentication & Role Management	Wed 28 Oct 2026	Tue 03 Nov 2026	5
T3	Event Management Module	Wed 04 Nov 2026	Tue 17 Nov 2026	10
T4	Budget Management Module	Wed 04 Nov 2026	Fri 13 Nov 2026	8
T5	Notifications & Role-Based Dashboard	Wed 18 Nov 2026	Tue 24 Nov 2026	5
T6	Integration, Testing & Deployment	Wed 25 Nov 2026	Fri 04 Dec 2026	6

Overall timeline: 19 Oct 2026 - 4 Dec 2026 (33 working days).

Exercise 6: Network-Based Scheduling

Develop a **Task-on-Node network diagram** using the activities, dependencies, and time estimates developed in the WBS and Gantt Chart.

Each task should be presented in a node and include:

- **Task ID**
- **Expected Time (E)**
- **ES - Earliest Start**
- **EF - Earliest Finish**
- **LS - Latest Start**
- **LF - Latest Finish**
- **Slack**

Students may use a different visual layout provided that all required information is clearly displayed. The team should clearly show the calculations used to obtain the scheduling values. Don’t forget to show slack time for each task.

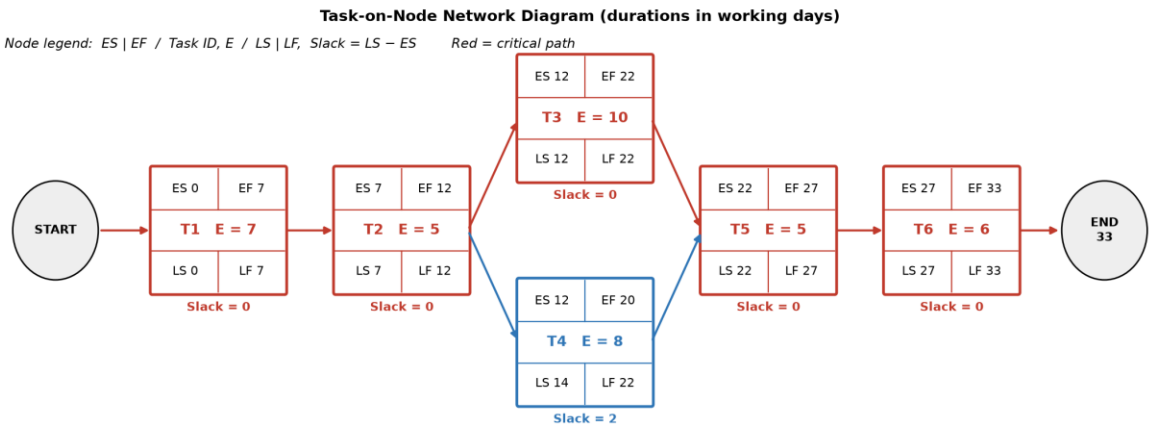


Figure 6: Task-on-Node Network Diagram

Calculations. Forward pass: ES = max(EF of predecessors), EF = ES + E. Backward pass: LF = min(LS of successors), LS = LF − E. Slack = LS − ES.

Task	E	ES	EF	LF	LS	Slack
T1	7	Project start = 0	0+7 = 7	LS(T2) = 7	7−7 = 0	0
T2	5	EF(T1) = 7	7+5 = 12	min(LS T3, LS T4) = min(12, 14) = 12	12−5 = 7	0
T3	10	EF(T2) = 12	12+10 = 22	LS(T5) = 22	22−10 = 12	0
T4	8	EF(T2) = 12	12+8 = 20	LS(T5) = 22	22−8 = 14	14−12 = 2
T5	5	max(EF T3, EF T4) = max(22, 20) = 22	22+5 = 27	LS(T6) = 27	27−5 = 22	0
T6	6	EF(T5) = 27	27+6 = 33	Project end = 33	33−6 = 27	0

Exercise 7: Critical Path

Using the completed network diagram, identify the **Critical Path** of the project. The Critical Path is the sequence of activities that determines the project completion time based on the network schedule. Activities on the Critical Path normally have Slack = 0.

The team must:

1. Identify the Critical Path.
2. List the activities forming the Critical Path.
3. Calculate the total expected project duration.
4. Explain why these activities form the Critical Path.
5. Verify the result using an appropriate scheduling/project-management tool.

Critical Path:

`T1 → T2 → T3 → T5 → T6`

Expected Project Duration:

```
7 + 5 + 10 + 5 + 6 = 33 days
```

Explanation:

Provide a brief explanation showing how the Critical Path was determined from the network calculations.

The network has two paths from start to end: T1 → T2 → T3 → T5 → T6 = 33 days and T1 → T2 → T4 → T5 → T6 = 31 days. The first path is the longest, and every task on it has ES = LS (Slack = 0), so any delay to one of these tasks delays the whole project. T4 is the only non-critical task: it finishes at day 20 but T5 cannot start until T3 finishes at day 22, which gives T4 2 days of slack.

Integration and Consistency Check

Before submission, the team must verify that all Lab 4 artifacts are consistent with one another and with previous labs.

Data Flow Modeling

- The Context Diagram represents the proposed system correctly.
- External entities are consistent with UML Use Case actors.
- Level-0 DFD uses the same relevant external entities.
- Context Diagram and Level-0 DFD are balanced.
- Level-0 contains at least five meaningful processes.
- Level-0 contains at least two relevant data stores.
- Core Processes Matrix contains the same major processes as the Level-0 DFD.
- DFD processes are consistent with Functional Requirements and Use Cases.

Project Scheduling

- WBS activities represent actual project work.
- Gantt Chart activities correspond to WBS activities.
- Task dependencies are logically defined.
- Time estimates are realistic and project-specific.
- Expected Times are correctly calculated.
- Network diagram reflects the WBS dependencies.
- ES, EF, LS, LF, and Slack calculations are correct.
- Critical Path is correctly identified.
- The Critical Path and project duration are verified using an appropriate scheduling/project-management tool.

Recommended Repository Structure

Update the same GitHub repository used in Labs 1-3:

```
Project-Repository/
│
├── README.md
│
├── Team/
│   ├── Team_Members.md
│   ├── Skills.md
│   ├── Contact_Information.md
│   └── CVs/
│
├── Project/
│   ├── Project_Title.md
│   ├── Project_Description.md
│   ├── Scope.md
│   ├── Process_Model.md
│   ├── Risk_Register.md
│   ├── Stakeholders.md
│   ├── Feasibility_Study.md
│   │
│   ├── Requirements/
│   │   ├── Scenarios.md
│   │   ├── Functional_Requirements.md
│   │   ├── Non_Functional_Requirements.md
│   │   ├── Use_Cases.md
│   │   └── Use_Case_Diagram/
│   │
│   ├── Data_Flow_Modeling/
│   │   ├── Context_Diagram/
│   │   ├── Level_0_DFD/
│   │   ├── Core_Processes_Matrix.md
│   │   └── DFD_Consistency_Check.md
│   │
│   └── Project_Scheduling/
│       ├── WBS.md
│       ├── Gantt_Chart/
│       ├── Network_Schedule/
│       └── Critical_Path.md
│
├── Documents/
│
└── Source_Code/
```

Deliverables

Each team must submit the following:

Data Flow Modeling

1. **Context Diagram**
2. **Level-0 DFD**
3. **Core Processes Matrix**
4. **DFD Consistency and Balancing Check**

Project Scheduling

5. **Work Breakdown Structure (WBS)**
6. **Gantt Chart**
7. **Task-on-Node network diagram**
8. **Time estimates**
9. **ES, EF, LS, LF, and Slack calculations**
10. **Critical Path identification**
11. **Expected project duration**
12. **Tool-based verification of the Critical Path and project duration**

Repository

13. Updated GitHub repository containing all required artifacts.
14. Meaningful commits showing team participation.

Progress Discussion

During the progress discussion, teams should be prepared to showcase and explain the data flow modeling of their project. Specifically, they should be able to answer:

- How the Context Diagram was derived from the Use Case model.
- Why the selected external entities correspond to the project's actors.
- How the Level-0 processes were identified.
- How the Core Processes Matrix relates to the DFD.

The lab instructor may ask individual team members to explain any part of the team's models or calculations.

Submission and Important Notes

- This is a **team-based submission**.
- Continue using the same project and GitHub repository established in previous labs.
- All diagrams must be created using a suitable software tool.
- Hand-drawn diagrams are not acceptable.
- The Context Diagram and Level-0 DFD must be balanced.
- The DFD must be consistent with the UML Use Case Diagram developed in Lab 3.
- The Core Processes Matrix must correspond to the Level-0 DFD.
- The WBS, Gantt Chart, and network schedule must represent the **same project activities**.
- The Critical Path calculations should be performed manually.
- Students should not simply copy generic software-development schedules. Activities, dependencies, and durations must be appropriate for their specific project.
- If Lab 4 reveals inconsistencies in the requirements, Use Cases, scope, or other previous artifacts, the relevant project documents should be updated.

- Students must follow the course AI policy and declare where and how AI tools were used in preparing the lab work.